SPARQL

RDF 查询语言 1.1 · 规范速查

SPARQL 1.1 — OVERVIEW · ONE LANGUAGE, ELEVEN DOCS

01 总览 · RDF 的查询语言

PART 1 · OVERVIEW

SPARQL:一条查询五段式,WHERE 里图模式组合出匹配,引擎按数据集求值,经协议返回结果;Update 让它读写对等。

查询骨架	PREFIX · SELECT · WHERE · 修饰 · 后处理	五段式
图模式	基本图 · FILTER · OPTIONAL · UNION	WHERE 的内功
执行	数据集 · 协议 · Update · 联邦 SERVICE	不止查询
语义与远方	蕴含档位开推理 · SPARQL 1.2 下一站	确定性规则

主线:图模式是内功:一切查询都是基本图模式 + 算子的组合。

本卡结构 = 本图:概览 → 查询核心 → 修饰与函数 → 执行与更新 → 生态与实操。

02 家族地图 · 一次发布十一份推荐标准

PART 1 · THE DOCUMENT FAMILY

SPARQL 之于 RDF,如 SQL 之于关系库。1.1 版于 **2013-03-21** 同日发布十一份 W3C Recommendation——查询只是入口,更新、协议、联邦、蕴含全套齐备。

文档	管什么
Query Language	查询语言本体(本文主线)
Update	增删改语言(§11)
Protocol	HTTP 绑定:query / update 参数
Graph Store HTTP Protocol	REST 式图管理(GET/PUT/POST/DELETE 整图)
Results JSON / CSV-TSV / XML	三种结果序列化(XML 是 1.0 第二版)
Federated Query	SERVICE 跨端查询
Service Description	端点自描述(能力、数据集)
Entailment Regimes	RDFS/OWL 蕴含下的查询语义

03 查询骨架 · 一条查询的五段式

PART 1 · QUERY ANATOMY

① 序言:PREFIX / BASE

② 数据集:FROM / FROM NAMED(可省,用端点默认)

③ WHERE { 图模式 } — 匹配发生在这里

④ 解的修饰:GROUP BY · HAVING · ORDER BY · 投影 · OFFSET/LIMIT

⑤ 结果形态:SELECT · CONSTRUCT · ASK · DESCRIBE

五种查询形态:SELECT 出变量表;CONSTRUCT 出**图**(模板重组);ASK 出真假;DESCRIBE 出关于某资源的子图(**由实现自定**——可移植性差的根源)。

```
# 骨架示例:谁在 2026 年发表过含 "kg" 的论文
PREFIX ex: <http://example.org/>
SELECT DISTINCT ?author ?title
WHERE {
  ?paper a ex:Paper ;
         ex:title ?title ;
         ex:author ?author .
  FILTER(CONTAINS(LCASE(?title), "kg"))
  ?paper ex:published ?date .
  FILTER(YEAR(?date) = 2026)
}
ORDER BY DESC(?date) LIMIT 20
```

04 图模式算子 · 组合的基本功

PART 2 · GRAPH PATTERN ALGEBRA

算子	写法	语义要点
基础图模式	三元组序列	同组内共享变量,全部匹配
并	`{ P1 } UNION { P2 }`	两模式解集取并——各自独立求值
可选	`OPTIONAL { P }`	左外连接:匹配不上留空变量;配 `BOUND(?v)` 判空
过滤	`FILTER(expr)`	只作用于**所在组**;EXISTS / NOT EXISTS 内可引用组内变量
差	`MINUS { P }`	减去可兼容的解——见下一节,与 NOT EXISTS 有别
绑定	`BIND(expr AS ?v)`	表达式赋值;不能覆盖已有变量
内联值表	`VALUES ?x { a b }`	解集硬编码;可 JOIN 进模式
子查询	`{ SELECT … WHERE {…} }`	先内后外;内层先做完修饰再 join

```
# OPTIONAL + 聚合:有人没邮箱也要出现在报表里
SELECT ?person (COUNT(?mail) AS ?mails)
WHERE {
  ?person a foaf:Person .
  OPTIONAL { ?person foaf:mbox ?mail }
}
GROUP BY ?person
```

05 MINUS vs NOT EXISTS · 一字千金的差别

PART 2 · MINUS VS NOT EXISTS

```
# 谁没有邮箱 —— 两种写法
# ① MINUS:对「已有解」做集合减法
SELECT ?p WHERE { ?p a foaf:Person
  MINUS { ?p foaf:mbox ?m } }
# ② NOT EXISTS:逐解判定,内层是过滤器
SELECT ?p WHERE { ?p a foaf:Person
  FILTER(NOT EXISTS { ?p foaf:mbox ?m }) }
```

差别在哪

MINUS 按**共享变量**相减:两模式没有共享变量时,MINUS 减不掉任何东西(全保留);NOT EXISTS 是逐解相关子查询,永远按内层模式判定。性能上 MINUS 常更优,语义上 NOT EXISTS 更直观——两边都要懂。

06 属性路径 · 把递归写进谓语位

PART 2 · PROPERTY PATHS

路径	写法	语义
谓词	`iri`	一步
反向	`^iri`	倒着走(被引用)
序列	`p1 / p2`	先…后…
可选其一	`p1 | p2`	二者任一
零或多	`p*`	含零步——自己到自己恒真
一或多	`p+`	传递闭包——找所有后代就靠它
零或一	`p?`	可有可无
取反集	`!(p1|p2)` · `!^p`	除了这些谓词
分组	`(p1/p2)*`	括号控优先级

```
# 组织架构树:找 ?boss 的全部下属(传递闭包)
?employee ex:reportsTo+ ?boss .
# 引用网:别人的引用别人的引用……但不经过自引
?src (cito:cites / cito:cites) ?dst .
# 反向一步 + 正向闭包混合
?paper ^ex:cites/(ex:author)? ?who
```

路径与 DISTINCT

p* / p+ 的解集按**路径存在性**给出节点对,多条路径只出一对;但 `*` 含零步,常与显式条件配合使用。复杂路径的执行代价因实现而异,大图上先 EXPLAIN。

07 聚合与分组 · 七个函数包打报表

PART 3 · AGGREGATES & GROUPING

聚合	作用	备注
`COUNT(*) / COUNT(?v) / COUNT(DISTINCT ?v)`	计数	* 数解,?v 数非空值
`SUM / AVG / MIN / MAX`	求和均值极值	数值型字面量
`GROUP_CONCAT(?v ; separator=", ")`	串接	**顺序不保证**——要排序先在子查询里 ORDER BY
`SAMPLE(?v)`	任意取一个	哪由实现定,别当确定性用

投影纪律

GROUP BY 之后,SELECT 里只能出现分组变量与聚合——「非聚合裸变量」直接报错,这是 SPARQL 对 SQL 用户的第一个下马威。

08 函数总表 · FILTER 里的全部武器

PART 3 · FUNCTION CATALOG

函数式形式与术语判断

`BOUND(?v) · IF(c,t,e) · COALESCE(…)`	判空 · 三元 · 首个非错	`EXISTS / NOT EXISTS`	子模式存在性
`sameTerm(a,b) · RDFterm-equal`	词项相等/一般相等	`IN(…) / NOT IN(…)`	成员判断
`isIRI isBlank isLiteral isNumeric`	术语类型四问	`str lang datatype`	拆字面量三件套
`IRI(s) · BNODE() · STRDT · STRLANG`	构造术语	`UUID() / STRUUID()`	生成唯一标识

字符串

`STRLEN SUBSTR UCASE LCASE`	`STRSTARTS STRENDS CONTAINS`	`STRBEFORE STRAFTER`	`CONCAT ENCODE_FOR_URI`
`langMatches(lang(?l),"zh")`	`REGEX(s,p[,f])`	`REPLACE(s,p,r[,f])`	正则用 XPath 语法,flags 里 "i" 最常用

数值 · 日期 · 哈希

`abs round ceil floor RAND` `now year month day hours minutes seconds timezone tz` `MD5 SHA1 SHA256 SHA384 SHA512`

错误语义

类型不匹配的 FILTER 表达式**报错即排除该解**——COALESCE 的价值正在于此:给可能出错的链兜底。布尔有效值规则(EBV)决定非布尔表达式怎么变真值,边界情形见规范 §11.2。

09 数据集与命名图 · 一次查多张图

PART 3 · DATASETS & NAMED GRAPHS

```
# FROM 定默认图,FROM NAMED 挂命名图
SELECT ?who ?g
FROM <http://example.org/default>
FROM NAMED <http://example.org/graph:alice>
FROM NAMED <http://example.org/graph:bob>
WHERE { GRAPH ?g { ?s foaf:knows ?who } }

# 不写 FROM:用端点默认配置;GRAPH ?g 遍历所有命名图
SELECT ?g ?s WHERE { GRAPH ?g { ?s ?p ?o } }
```

两组关键词 scope

- `FROM / FROM NAMED`:组成查询数据集——不匹配的图被排除在考虑之外
- `GRAPH ?g { … }`:在命名图内匹配;变量 ?g 绑定图名

与 RDF 1.1 数据集 差异

- SPARQL 的图名**只许 IRI**,RDF 1.1 允许 bnode(引 PL.01 §07)——先 Skolem 化
- 端点默认数据集由实现决定:查之前先看 Service Description

为什么要有命名图

溯源分账、多版本共存、权限隔离——都靠「哪条三元组属于哪张图」。SPARQL Update 与 Graph Store 协议同样按图操作。

10 蕴含档位 · 查询时推理开几档

PART 4 · ENTAILMENT LEVELS

档位	查询语义
simple	纯子图匹配,无推理(默认)
RDFS	类/属性层级、domain/range 全部生效(引 PL.02)
OWL-Direct / OWL-RDF-Based	OWL 公理参与匹配;Direct 限 DL 子集,推理开销大

开档要谨慎

蕴含档让结果集暴涨(层级上浮全进来),且各端点支持的档位不同——先查 Service Description,按需在请求头里声明档位。

11 Update · 全部八类操作

PART 4 · UPDATE

类	操作	一句话
图更新	`INSERT DATA {…}` / `DELETE DATA {…}`	按字面三元组增删(不允许变量)
	`DELETE {…} INSERT {…} WHERE {…}`	按模式删旧插新——变量来自 WHERE
图管理	`LOAD <iri> [INTO GRAPH <g>]`	远程拉图入库([SILENT] 吞错)
	`CLEAR GRAPH <g>` / DEFAULT / NAMED / ALL	清空图内容
	`CREATE / DROP GRAPH <g>`	建图 / 连图带定义删掉
图搬运	`COPY / MOVE / ADD` [SILENT] FROM→TO	整图复制/搬移/并入(目标原图被替换或保留语义见规范)

```
# 归档:把 2025 图的旧论文移进 archive 图
DELETE { GRAPH :papers2025 { ?p ?o ?v } }
INSERT { GRAPH :archive   { ?p ?o ?v } }
WHERE  { GRAPH :papers2025 { ?p ex:year "2025"^^xsd:gYear ; ?o ?v } } ;

COPY SILENT :staging TO :production .
```

Graph Store HTTP Protocol

不写 SPARQL 也能管图:HTTP PUT/POST/DELETE/GET 直接对 `?graph=` 或 `graph=` 参数代表的整图操作——ETL 脚本最爱。

12 协议与结果格式 · HTTP 面孔

PART 4 · PROTOCOL & RESULTS

操作	HTTP 形态
query	GET/POST(form)到端点;Accept 协商结果格式
update	POST(form)到 update 端点——规范要求 update 走 POST
结果格式	`application/sparql-results+json` · `+xml` · `text/csv` · `text/tab-separated-values`;CONSTRUCT/DESCRIBE 出 RDF 序列化(Turtle 等)

```
# Service Description:端点自我介绍(GET 端点根,内容协商)
<> a sd:Service ;
    sd:endpoint <http://example.org/sparql/> ;
    sd:supportedLanguage sd:SPARQL11Query, sd:SPARQL11Update ;
    sd:defaultEntailmentRegime rdfs:RDFS .
```

13 联邦查询 · SERVICE 与分布式 joins

PART 5 · FEDERATED QUERY

```
# 把 Wikidata 的片段拉进本地模式
SELECT ?label ?country WHERE {
  :alice foaf:based_near ?city .
  SERVICE <https://query.wikidata.org/sparql> {
    ?city rdfs:label ?label .
    ?city wdt:P17 ?country .
    FILTER(LANG(?label) = "zh")
  }
}
```

联邦的代价

SERVICE 是黑盒:过滤下推靠各端点自觉,慢查询是常态。把可下推的 FILTER 写进 SERVICE 块内部、必要时用 VALUES 预绑关键变量,能省一个数量级。

14 SPARQL 1.2 · 下一站

PART 5 · SPARQL 1.2 NEXT

正在路上(检索于 2026-10)

SPARQL 1.2 由同一工作组推进(十二份文档,引 PL.06 §02):核心看点是与 RDF 1.2 对齐——三元词项可进查询模式、方向性语言标签的函数支持、Canonical SPARQL(规范化查询形)。本文的 1.1 内容全部是其子集。

15 实操坑清单

PART 5 · PITFALLS

01 OPTIONAL 里套 FILTER:过滤留在 OPTIONAL 组**内部**才按预期工作,挪到外层语义就变。

02 MINUS 无共享变量 = 白减;该用 NOT EXISTS 的场景别图省事。

03 GROUP BY 后 SELECT 只许分组变量与聚合——裸变量直接报错。

04 GROUP_CONCAT 顺序不保证:要有序,先子查询排序再聚合。

05 OFFSET 翻页必须配 ORDER BY,否则页与页会重叠漏数据。

06 p* 恒含零步(自己到自己),要「严格后代」用 p+。

07 DESCRIBE 的子图范围由实现自定——跨端点结果不可比,生产查询写 SELECT。

08 字面量比较认类型:"5" ≠ 5;数值过滤前先 FILTER(datatype) 或统一类型。

09 语言标注串比较用 langMatches(lang(?v),"zh"),直接 = "zh" 会漏掉 zh-CN。

10 Update 默认无事务保证(端点自行决定):大批改动拆批 + 校验,别指望回滚。

11 SERVICE 里写死大模式:下推过滤、VALUES 预绑,否则联邦查询慢出天际。

12 空 WHERE / 空模式在 update 里是合法边界(全图操作),手滑写空的 DELETE WHERE = 清库。